

# Ulsan-Fault earthquake catalog pipeline

Reference manual: detection → association → location → relocation (HypoDD dt.ct + dt.cc)

Long-term (2010–present) catalog for the Ulsan Fault, SE Korea

src/ufpipe + detection\_test

ufpipe (src/ufpipe) is the **end-to-end pipeline**: six stages, detection → association → augment → PHS → locate → **relocate** (HypoDD dt.ct + dt.cc). The final **relocate** stage produces the double-difference relocated catalog. The heavy dt.cc/xcorr/HypoDD *engine* is external (15.PocketQuake korea-cluster-relocation, HypoDD v2.1beta); ufpipes orchestrates it. `detection_test/` is now just the 4-picker-comparison pilot plus the shared per-month detection/association *feeder* that the relocate stage consumes (see §7). The relocation internals additionally have a dedicated guide: `detection_test/reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`.

## Contents

|          |                                                                                   |          |
|----------|-----------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>What the pipeline does</b>                                                     | <b>1</b> |
| 1.1      | ufpipe vs detection_test . . . . .                                                | 2        |
| 1.2      | Two orthogonal “model” dimensions . . . . .                                       | 2        |
| <b>2</b> | <b>Install &amp; environment</b>                                                  | <b>2</b> |
| <b>3</b> | <b>Stages 1–5: detection through location</b>                                     | <b>3</b> |
| 3.1      | Source of truth for pick↔event assignment . . . . .                               | 3        |
| <b>4</b> | <b>Parameters (src/ufpipe/config.py — the single disclosed source)</b>            | <b>3</b> |
| 4.1      | Detection / DSP . . . . .                                                         | 4        |
| 4.2      | Association (PyOcto) — daily-chunked, ASSOC_GATE (lenient) vs ASSOC_GATE_STRICT . | 4        |
| 4.3      | Velocity model & location . . . . .                                               | 4        |
| <b>5</b> | <b>How to run</b>                                                                 | <b>5</b> |
| 5.1      | Running each stage separately . . . . .                                           | 5        |
| 5.2      | Relocation — running the last stage (stage 6) . . . . .                           | 6        |
| <b>6</b> | <b>Outputs &amp; layout</b>                                                       | <b>6</b> |
| <b>7</b> | <b>Stage 6: relocation — HypoDD dt.ct + dt.cc</b>                                 | <b>7</b> |
| <b>8</b> | <b>Robustness &amp; invariants</b>                                                | <b>8</b> |
| <b>9</b> | <b>Troubleshooting / provenance checks</b>                                        | <b>8</b> |

## 1 What the pipeline does

Build an **internally consistent, long-term earthquake catalog** for the Ulsan Fault region (SE Korea), 2010–present, from continuous seismic waveforms, using AI phase pickers so small events are detected uniformly even as the recording network grows, and finally **relocate** the catalog with double-difference (HypoDD dt.ct + dt.cc) for sub-100 m relative precision. Every stage is scripted and parameterized; results reproducible.

| Detection            | Association            | Location           | QC                    | Relocation           |
|----------------------|------------------------|--------------------|-----------------------|----------------------|
| PhaseNet / PhaseNet+ | PyOcto (+augment)      | HYPOINVERSE        | uf_cluster            | HypoDD dt.ct + dt.cc |
| <i>picks</i> →       | <i>events+assign</i> → | <i>.sum/.arc</i> → | <i>well-located</i> → | <i>hypoDD.reloc</i>  |

## 1.1 ufpipes vs detection\_test

ufpipe is the production end-to-end pipeline. `detection_test` is the pilot experiment (the 2016 4-picker comparison) and also hosts two things ufpipes’s `relocate` stage reuses: the per-month, multi-network detection/association *feeder* (`detection_test/lib/`) and the validated relocation orchestrator (`run_picker_reloc.py`) that drives the external HypoDD engine.

|         | ufpipe (src/ufpipe)                                                | detection_test                                                                                      |
|---------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Role    | the end-to-end catalog + relocation pipeline                       | 4-picker-comparison pilot + shared feeder/orchestrator                                              |
| Stages  | detection → association → augment → PHS → locate → <b>relocate</b> | per-month detection/association ( <code>lib/</code> ); reloc driver ( <code>reloc_2016_uf/</code> ) |
| Driver  | <code>python -m ufpipes.run_pipeline</code>                        | (called by ufpipes’s relocate stage)                                                                |
| Ends at | HypoDD dt.cc (§7)                                                  | feeds the relocate stage                                                                            |

ufpipes’s own detection + association now cover **KS/KG/GJ/NS** with daily-chunked association (§5). The `relocate` stage nonetheless still consumes `detection_test/lib`’s per-month outputs as its feeder (unchanged and validated); converging relocate onto ufpipes’s own association is future work. See §7.

## 1.2 Two orthogonal “model” dimensions

| Dimension             | Flag                    | Values                                                                                           |
|-----------------------|-------------------------|--------------------------------------------------------------------------------------------------|
| <b>Picker model</b>   | <code>--model</code>    | <code>stead</code> , <code>original</code> , <code>phasenet_plus</code> , <code>eqt</code> , ... |
| <b>Velocity model</b> | <code>--velmodel</code> | <code>kim1983</code> , <code>kim2011</code>                                                      |

Any picker model can be located with any velocity model. `stead/original` are SeisBench PhaseNet weights; `phasenet_plus` is EQNet PhaseNet+ (needs a local EQNet clone).

## 2 Install & environment

### One-time setup

1. `conda env create -f environment.yml && conda activate ulsan` (the code also runs in base here)
2. `pip install torch --index-url https://download.pytorch.org/whl/cu128` (match your CUDA; GPU strongly recommended)
3. `pip install -e .` (installs the `uflib` + `ufpipes` packages, editable)
4. **HYPOINVERSE**: the external `hyp1.40` binary must be on `PATH` (not pip-installable).
5. **PhaseNet+ only**: a local EQNet clone at `config.EQNET_DIR`.

After `pip install -e .`, `import ufpipes.config` and `python -m ufpipes.run_pipeline` work from any directory — no `sys.path` hacks.

### 3 Stages 1–5: detection through location

STAGES = [detection, association, augment, phs, locate, relocate] — run in order. Stages 1–5 (below) are functions in `src/ufpipe/core.py`; the relocate stage (6) is `src/ufpipe/relocate.py` and has its own §7. `run_pipeline.py` chains them per year.

| # | Stage              | In → Out                                            | What it does                                                                                                                                                                                                                               |
|---|--------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>detection</b>   | continuous mSEED → picks/<br>(per station-day)      | AI picker on each station-day across <b>KS/KG/GJ/NS</b> (per-year multi-network table, <code>stations.py</code> ); anti-alias >100 Hz, unify mixed rates, resample to 100 Hz, pick P/S above threshold. GPU. Idempotent (skips done days). |
| 2 | <b>association</b> | picks → <code>pyocto_*.csv</code> + assignment      | PyOcto octree associator groups picks into events (gate below), <b>daily-chunked</b> ( $\pm 150$ s/day, in-day origins) so it scales to the dense NS array. Coords from the multi-network table.                                           |
| 3 | <b>augment</b>     | assignment → augmented assignment                   | scan daily picks for orphans PyOcto missed, add to each event so HYPOINVERSE sees the full pick set. Backs up + overwrites in place; idempotent.                                                                                           |
| 4 | <b>phs</b>         | assignment → <code>UF{year}.phs</code>              | write the HYPOINVERSE phase file from the (augmented) assignment.                                                                                                                                                                          |
| 5 | <b>locate</b>      | <code>.phs</code> → <code>UF{year}.{sum,arc}</code> | run HYPOINVERSE ( <code>hyp1.40</code> ) with the chosen velocity model → located catalog + errors.                                                                                                                                        |

#### Pipeline order matters

**augment** runs *between* **association** and **phs**, so HYPOINVERSE sees the full augmented pick set and computes correct GAP/DMIN/ERZ/RMS. Do not reorder.

#### 3.1 Source of truth for pick↔event assignment

The **PyOcto assignment (after augmentation)** is authoritative for which (`station`, `phase`) tuples belong to each event. Do NOT use a legacy time-window pick dump — when two events fall <60 s apart at the same station the earliest pick “wins” and leaks into the wrong event. Downstream (SAC-header writers, magnitude/QC notebooks) must read `pyocto_assignment_{velmodel}_{year}.csv`.

### 4 Parameters (`src/ufpipe/config.py` — the single disclosed source)

Change defaults here. Everything is one file.

## 4.1 Detection / DSP

| Parameter                   | Value                               | Meaning                                             |
|-----------------------------|-------------------------------------|-----------------------------------------------------|
| SAMPLING_RATE               | 100.0 Hz                            | the pickers' trained rate; all data resampled to it |
| ANTIALIAS                   | lowpass 45 Hz, 8 corners, zerophase | applied BEFORE down-sampling (>100 Hz KG/NS)        |
| BANDPASS                    | 1–40 Hz, 4 corners, causal          | legacy detection path; PhaseNet+ runs raw           |
| P_THRESHOLD,<br>S_THRESHOLD | 0.2                                 | SeisBench pick probability threshold                |
| PNPLUS_MIN_PROB             | 0.3                                 | EQNet PhaseNet+ pick threshold                      |
| PNPLUS_NT                   | 1024×36                             | PhaseNet+ patch length (whole-day, cut_patch)       |
| MAX_CORES                   | 24                                  | CPU cap (respects taskset affinity)                 |

## 4.2 Association (PyOcto) — daily-chunked, ASSOC\_GATE (lenient) vs ASSOC\_GATE\_STRICT

Association runs **one calendar day at a time**: a  $\pm$ ASSOC\_OVERLAP\_S (150 s) window per day is associated and only events whose *origin* falls in-day are kept (dedup). This is physically equivalent to a single pass (local events are seconds long) but keeps each PyOcto solve to seconds — a whole-year single pass is intractable on the dense  $\sim$ 200-station NS array ( $\gg$ 1 h, 12 GB). Station coordinates come from the multi-network year table (`stations.py`), so KS/KG/GJ/NS all associate.

| Parameter (config)      | lenient                          | _STRICT | Meaning                            |
|-------------------------|----------------------------------|---------|------------------------------------|
| REGION_CENTER $\pm$ pad | (35.856,129.224) $\pm$ (1.0,1.2) |         | search area (deg)                  |
| ASSOC_ZLIM              | (0, 30)                          |         | depth range (km)                   |
| ASSOC_TIME_BEFORE       | 300                              |         | origin-search window (s)           |
| ASSOC_OVERLAP_S         | 150                              |         | daily-chunk overlap (s)            |
| n_picks                 | 4                                | 6       | min total picks                    |
| n_p / n_s               | 2 / 2                            | 3 / 3   | min P / S picks                    |
| n_ps                    | 1                                | 2       | min stations with both P+S (depth) |
| ASSOC_PICK_MATCH_TOL    | 1.5                              |         | residual cap (s)                   |

Pass `--strict` to use ASSOC\_GATE\_STRICT (fewer, cleaner events). `--workers N` parallelises the independent daily chunks. Velocity for association = ASSOC\_VELMODEL (kim2011 1-D), matching the reloc feeder. Output schema (events `idx,time,x,y,z,picks,latitude,longitude,depth`; assignment `event_idx,pick_idx,residual,station,phase,time`) is unchanged, so `phs/locate/relocate` consume it as before.

## 4.3 Velocity model & location

DEFAULT\_VELMODEL = kim2011; kim1983 also available (`data/metadata/velocity_model/kim1983.csv`, a layered  $V_p/V_s$  model). PHASE\_CHANNELS = {P: HHZ, S: HHN}.

## 5 How to run

### Standard runs (from the repo root)

|                                                                                                                |                                       |
|----------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>python -m ufpipes.run_pipeline --model original --years 2024</code>                                      | one year, all 5 stages                |
| <code>python -m ufpipes.run_pipeline --model original --years 2010-2024</code>                                 | the whole record                      |
| <code>python -m ufpipes.run_pipeline --model original --years 2015,2016 --velmodel kim1983</code>              | specific years, kim1983               |
| <code>python -m ufpipes.run_pipeline --model original --years 2024 --stage-from association</code>             | resume (picks already done)           |
| <code>python -m ufpipes.run_pipeline --model original --years 2024 --days 1-3</code>                           | detection on a few days (testing)     |
| <code>python -m ufpipes.run_pipeline --model original --years 2024 --strict</code>                             | strict association gate               |
| <code>python -m ufpipes.run_pipeline --model original --years 2016 --stage-from relocate --through dtcc</code> | relocation only (dt.cc; needs feeder) |

**CLI flags:** `--model` · `--years` ('2010-2024' or '2015,2016') · `--velmodel` · `--days` · `--device` {cuda,cpu} · `--workers` · `--strict` · `--force` (allow writing to stead) · `--stage-from` {detection,association,augment,phs,locate} (start mid-chain).

### Shared 64-core server — always pin cores

Detection sizes its preprocessing pool and torch threads from the process CPU *affinity*, capped by `MAX_CORES=24`. Launch under `taskset` so the whole job stays within budget:

```
OMP_NUM_THREADS=1 taskset -c 0-7 python -m ufpipes.run_pipeline --model original --years 2010-2024
```

Without pinning, PhaseNet+ grabbed ~49 cores / 193 threads and starved everything. Inference is **GPU-preferred** (warns loudly, never silently falls back to CPU).

Detection is **idempotent** — it skips station-days whose picks already exist, so runs resume safely after an interruption.

### 5.1 Running each stage separately

Every `ufpipe` stage is also a standalone CLI (`--model --year`), so you can run detection alone, then association alone, etc. Order is `detection` → `association` → `augment` → `phs` → `locate`. Each reads the previous stage's output from disk.

| Stage       | standalone command                                                                                                            |
|-------------|-------------------------------------------------------------------------------------------------------------------------------|
| Detection   | <code>python -m ufpipes.detection --model original --year 2024<br/>[--networks KS,KG,GJ,NS] [--days 1-5] [--workers 8]</code> |
| Association | <code>python -m ufpipes.association --model original --year 2024<br/>[--networks KS,KG,GJ,NS] [--workers 8] [--strict]</code> |
| Augment     | (runs inside <code>run_pipeline</code> ; or call <code>core.run_augment_year</code> from Python)                              |
| PHS         | <code>python -m ufpipes.make_phs --model original --year 2024</code>                                                          |
| Location    | <code>python -m ufpipes.run_hypoinverse --model original --year 2024<br/>--velmodel kim2011</code>                            |

Or drive the same stage functions from Python / a notebook:

```
from ufpipes import core, config
core.run_detection_year("original", 2024, days=range(1,6))
events, assign = core.run_association_year("original", 2024)
core.write_phs("original", 2024); core.run_hypoinverse_year("original", 2024,
velmodel="kim2011")
```

## 5.2 Relocation — running the last stage (stage 6)

Run the relocate stage via `ufpipe` (`--stage-from relocate`); it wraps the validated year-general orchestrator. Its **feeder** — the per-month, KS/KG/GJ/NS detection + association — is generated separately in `detection_test/` (the dense NS array needs daily-chunked association). If a year's feeder is missing, the relocate stage preflights and prints exactly what to run. The per-step feeder + reloc commands:

| Step          | command                                                                                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Readiness     | <code>python preflight_year.py --year 2016</code>                                                                                                               |
| Detection     | <code>python ../lib/run_seisbench_picker.py --model original --month<br/>2016-01 (loop months; PN+: run_pnplus_month.py)</code>                                 |
| Association   | <code>python ../lib/associate_daily.py --picker original --month<br/>2016-01</code>                                                                             |
| Reloc (to QC) | <code>python run_picker_reloc.py --picker original --year 2016<br/>--through hypoinverse (fast; SAC→HYPOINVERSE→QC)</code>                                      |
| Reloc (full)  | <code>python run_picker_reloc.py --picker original --year 2016<br/>--through dtcc --clean-cache (+ rereference→xcorr→HypoDD<br/>dt.cc; xcorr ~6 h dense)</code> |

`--through hypoinverse` stops at the QC'd absolute-location subset; `--through dtcc` continues to the double-difference relocation. `--clean-cache` deletes the ~tens-of-GB interp cache after `dt.cc` (essential for multi-year runs). Full detail: `PIPELINE.md` + `study_guide.pdf`.

## 6 Outputs & layout

All pipeline products go under `outputs/models/<model>/` (regenerable; gitignored):

| Path                                         | Contents                                           |
|----------------------------------------------|----------------------------------------------------|
| .../detection_location/<year>/picks/         | per-station-day picks (stage 1)                    |
| .../pyocto/pyocto_{vm}_{year}.csv            | associated events (stage 2)                        |
| .../pyocto/pyocto_assignment_{vm}_{year}.csv | pick→event assignment (authoritative)              |
| .../station_table/stations_{year}.csv        | per-year station roster (discovered from metadata) |
| .../HypoInv/PHS/UF{year}.phs                 | HYPOINVERSE phase file (stage 4)                   |
| .../HypoInv/{vm}/UF{year}.{sum,arc,prt}      | located catalog + errors (stage 5)                 |

**Inputs** (restructured 2026-07): waveforms in `KS_KG/` (station dirs at root, parallel to `GJ/NS/`); station metadata in `data/metadata/ks_kg_station_table/`; velocity model in `data/metadata/velocity_model/`; HYPOINVERSE control inputs (`STA/kim*`) in `data/hypoinv/`.

## 7 Stage 6: relocation — HypoDD `dt.ct` + `dt.cc`

The 6th ufpipeline stage: turn the QC'd absolute-location catalog into a **double-difference relocated** catalog for sub-100 m relative precision.

```
python -m ufpipeline.run_pipeline --model original --years 2016 --stage-from relocate
--through dtcc --clean-cache
```

`ufpipe.relocate` (`src/ufpipe/relocate.py`) is a *thin wrapper*: it pre-flights the per-month feeder inputs then invokes the validated orchestrator `detection_test/reloc_2016_uf/run_picker_reloc.py`, which drives the external HypoDD/xcorr engine. Internal stages (after the per-month detection + association feeder, `KS/KG/GJ/NS`):

| # | Stage        | What it does                                                                                                                                                                       |
|---|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | SAC store    | event-idx-keyed native-rate SAC waveforms from the associated picks ( <code>event_sac/</code> )                                                                                    |
| 2 | HYPOINVERSE  | absolute location of all UF-box events ( <code>kim2011</code> ) → <code>.sum</code>                                                                                                |
| 3 | QC           | <code>uf_cluster.QC</code> : keep <code>erh&lt;5</code> , <code>erz&lt;5</code> , <code>gap&lt;270</code> , <code>num&gt;5</code> , <code>rms&lt;1.0</code>                        |
| 4 | re-reference | stamp each SAC's origin from the (full-run) HYPOINVERSE <code>.sum</code>                                                                                                          |
| 5 | ph2dt        | build <code>event.dat</code> + catalog differential times <code>dt.ct</code>                                                                                                       |
| 6 | xcorr (GPU)  | cross-correlation differential times <code>dt.cc</code> (band 5–20 Hz, interp 1000 Hz, <code>cc≥0.7</code> )                                                                       |
| 7 | HypoDD       | <code>kim2011</code> , <code>ISTART=2</code> , <code>ISOLV=2</code> (LSQR), adaptive damping (CND 40–80), HypoDD v2.1beta ( <code>MAXDATA=15M</code> ) → <code>hypoDD.reloc</code> |

### Relocation outputs (symlinked into the working dir)

Each run publishes `detection_test/reloc_<year>_uf[_<p>]/results/` with symlinks to the external run tree: `hypoDD.reloc.dtcc` (`dt.ct+dt.cc`), `hypoDD.reloc.dtct` (`dt.ct`-only), `HypoInv.full.sum`, ...“`dt.cc-resolved`” = events keeping  $\geq 1$  surviving cross-correlation link — the highest-precision subset.

**The invariant that matters (see PIPELINE.md)**

Origins flow into dt.cc: **rereference** stamps SAC origins from a `.sum`, and `xcorr` stores  $dt = (t_1 + \text{shift} - ot_1) - (t_2 - ot_2)$  — subtracting those origins. Use the *full-run* HYPOINVERSE solution QC gated on (never a redundant re-run), or the dt.cc *values* are corrupted. This is enforced by `run_picker_reloc.py::inject_full_hypoinverse`.

## 8 Robustness & invariants

- **Network growth is automatic.** Stations are *discovered from StationXML + on-disk coverage* each month; the deployed network for that period is what enters the run. No hard-coded station lists.
- **Mixed sampling rates handled.** >100 Hz stations (KG/NS at 200 Hz) are anti-alias-lowpassed then resampled to the pickers' 100 Hz. The anti-alias filter is applied BEFORE down-sampling (never after).
- **Velocity model, thresholds, and gates are epoch-invariant by choice**  $\Rightarrow$  the catalog stays internally comparable across years.
- **Idempotent & resumable** at every stage; `--stage-from` restarts mid-chain.
- **Do not edit the *stead* reference run** (it is the baseline). Scripts refuse `--model stead` writes unless `--force`.

## 9 Troubleshooting / provenance checks

| Symptom                                   | Check                                                                                                                                                                                                        |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detection finds no waveforms              | <code>python -c "import ufpipes.config as c,os; print(c.CONTINUOUS, os.path.isdir(c.CONTINUOUS))"</code> — must point at KS_KG/ and exist.                                                                   |
| <code>ImportError: ufpipes / uflib</code> | <code>pip install -e .</code> in the active env; <code>import ufpipes.config; print(__file__)</code> must be <code>src/ufpipes</code> .                                                                      |
| Wrong pipeline imported                   | <code>ufpipes</code> is renamed precisely to avoid the collision with the external korea-cluster-relocation <code>pipeline</code> . Verify <code>ufpipes.config.__file__</code> is under <code>src/</code> . |
| Events look over/under-associated         | compare <code>REGION</code> vs <code>--strict</code> ; the <code>pick_match_tolerance</code> is the primary knob.                                                                                            |
| GPU not used                              | inference is GPU-preferred and warns loudly; check the torch/CUDA install (never silently CPU).                                                                                                              |
| Sparse early years, few events            | expected — KS/KG-only network (2010–2016) has larger azimuthal gaps; the QC gate honestly reflects what the network resolves.                                                                                |

---

Companion docs: docs/overview.md, docs/pipeline.md, docs/how-to-run.md,  
docs/directory-structure.md · src/ufpipes/README.md · relocation pipeline:  
detection\_test/reloc\_2016\_uf/PIPELINE.md + study\_guide.pdf.